For each instance method like Pry#memory_size, there is a corresponding class method (Pry.memory_size) that returns the default value of the attribute:

```
Pry.memory_size       # => 100
```

Let's look a little deeper inside the Pry source code. To configure a Pry instance, you can call a method named Pry#refresh. This method takes a hash that maps attribute names to their new values:

```
pry.refresh(:memory_size => 99, :quiet => false)
pry.memory_size       # => 99
pry.quiet             # => false
```

Pry#refresh has a lot of work to do: it needs to go through each attribute (such as self.memory_size); initialize the attribute with its default value (such as Pry.memory_size); and finally check whether the hash argument contains a new value for the same attribute, and if it does, set the new value. Pry#refresh could do all of those steps with code like this:

```
def refresh(options={})
  defaults[:memory_size] = Pry.memory_size
  self.memory_size = options[:memory_size] if options[:memory_size]

  defaults[:quiet] = Pry.quiet
  self.quiet = options[:quiet] if options[:quiet]
  # same for all the other attributes...
end
```

Those two lines of code would have to be repeated for each and every attribute. That's a lot of duplicated code. Pry#refresh manages to avoid that duplication, and instead uses *Dynamic Dispatch (49)* to set all the attributes with just a few lines of code:

gems/pry-0.9.12.2/lib/pry/pry_instance.rb

```
def refresh(options={})
  defaults   = {}
  attributes = [ :input, :output, :commands, :print, :quiet,
                 :exception_handler, :hooks, :custom_completions,
                 :prompt, :memory_size, :extra_sticky_locals ]

  attributes.each do |attribute|
    defaults[attribute] = Pry.send attribute
  end
  # ...
  defaults.merge!(options).each do |key, value|
    send("#{key}=", value) if respond_to?("#{key}=")
  end

  true
end
```